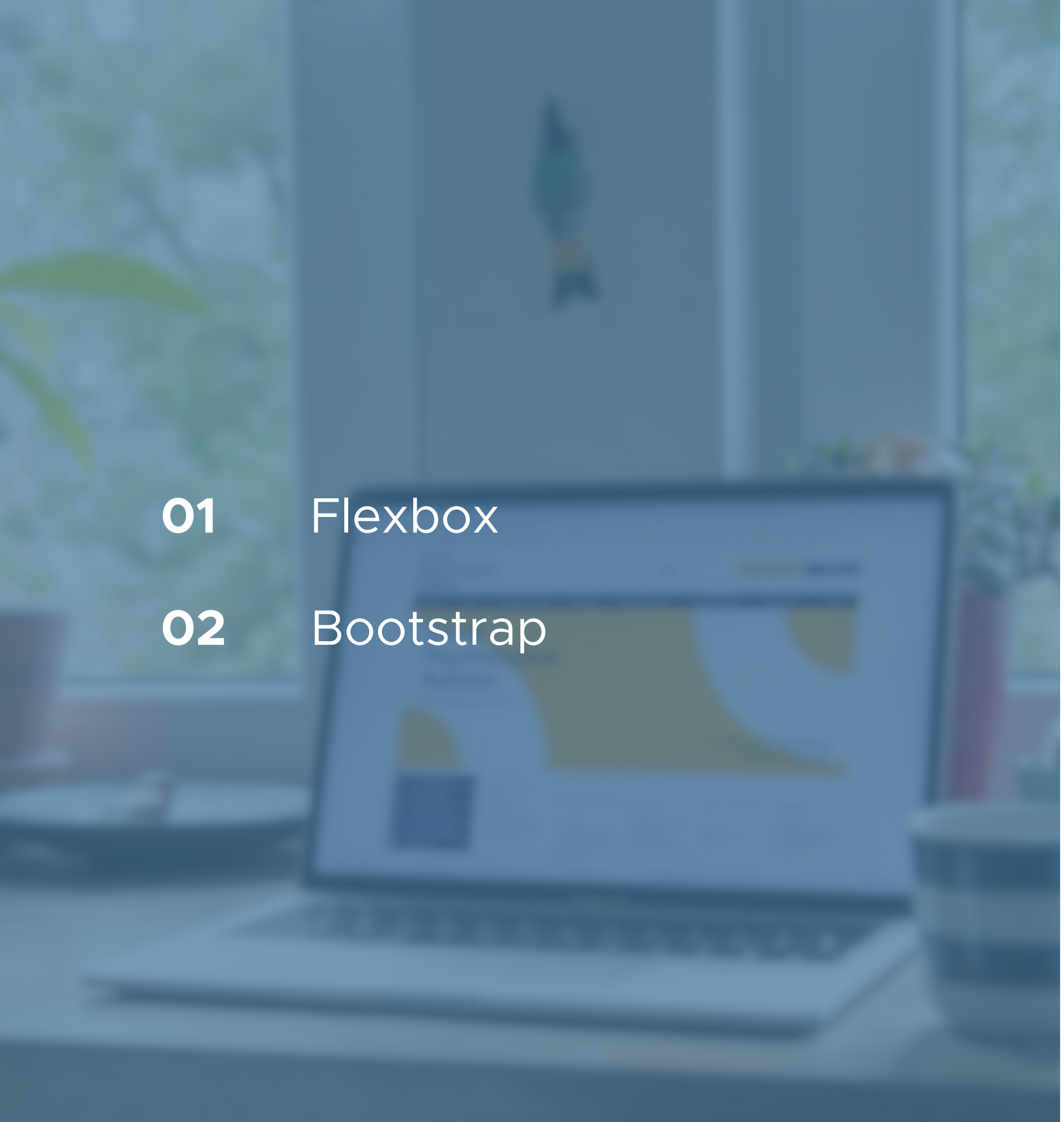
Full Stack Development Course

Eng. Elias Al Zaghrini

Flexbox & Bootstrap

Chapter 2



Flexbox & Bootstrap

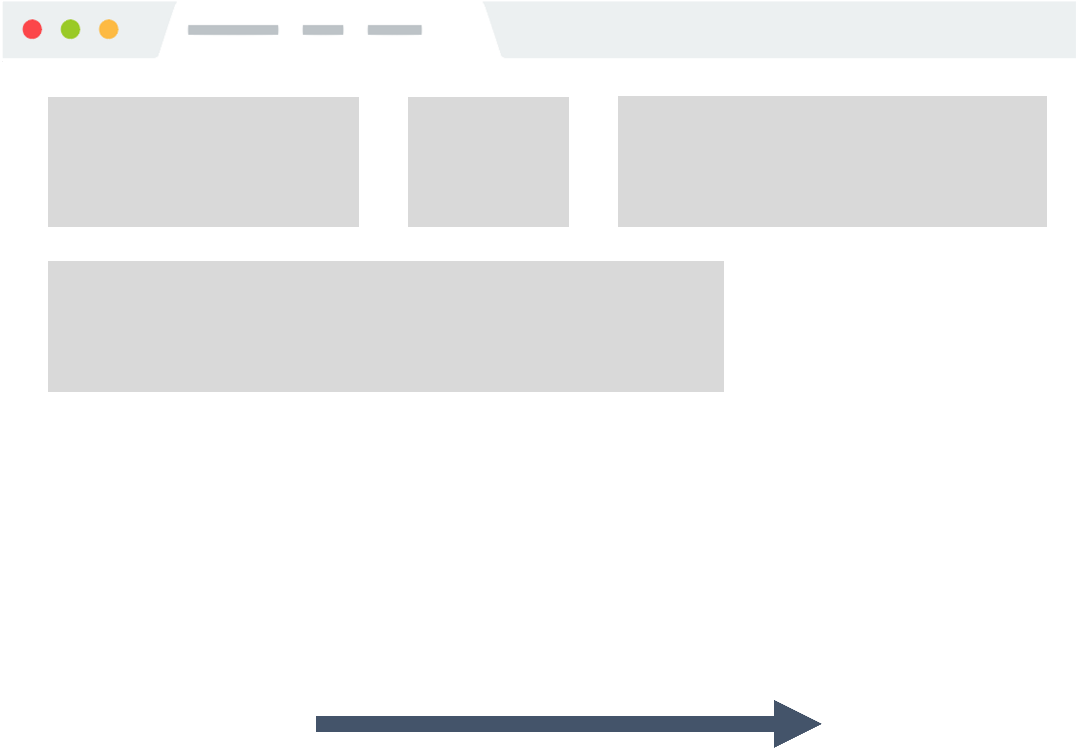
Chapter 2

Flexbox

Introduction

Block Layout

- Laying out large
- Sections of a page



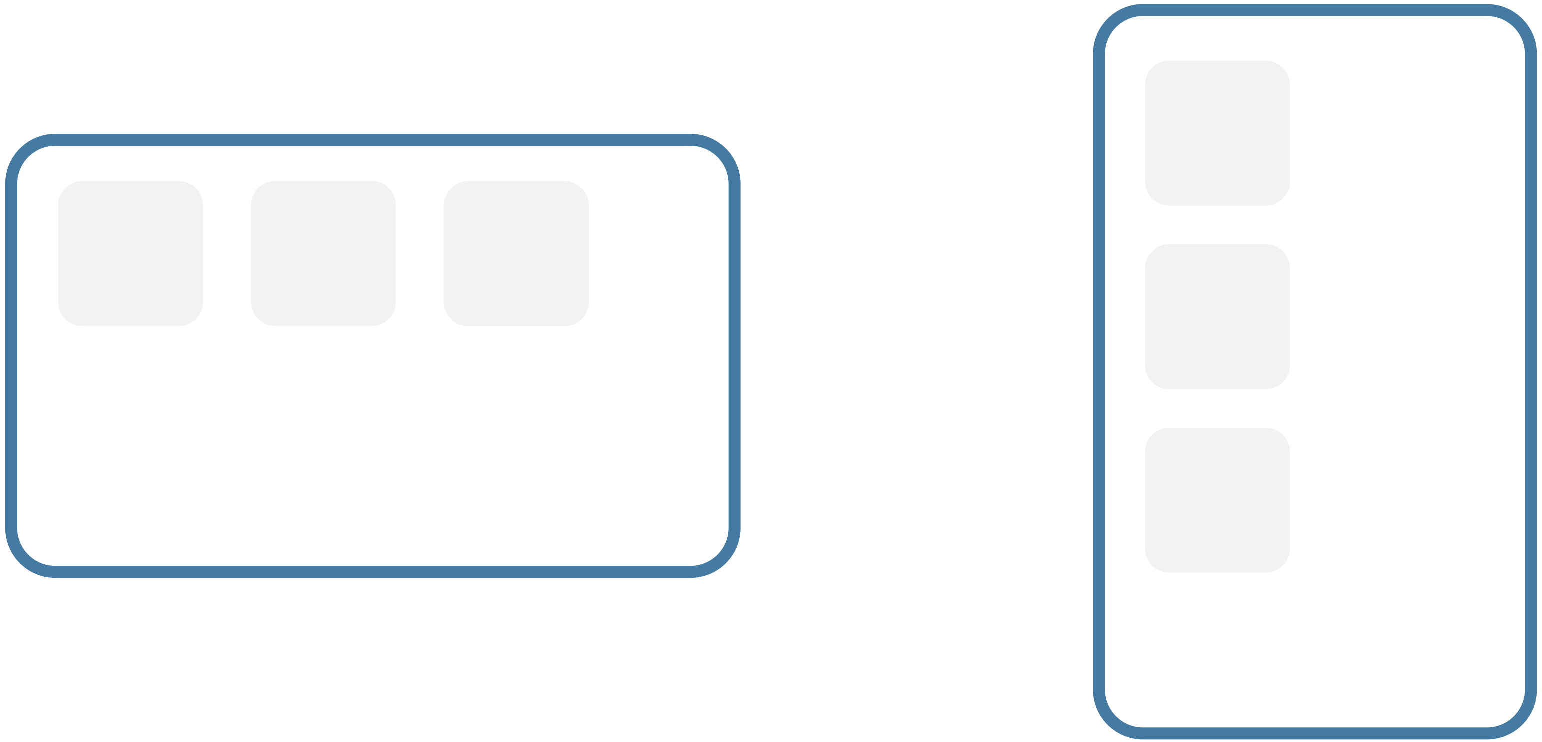
Inline Layout

- Laying out text and other inline content within a section

Flexbox

Introduction

- To achieve more complicated layouts, enable a different CSS layout rendering mode: Flex layout.
- Flex layout defines a special set of rules for laying out items in rows or columns.

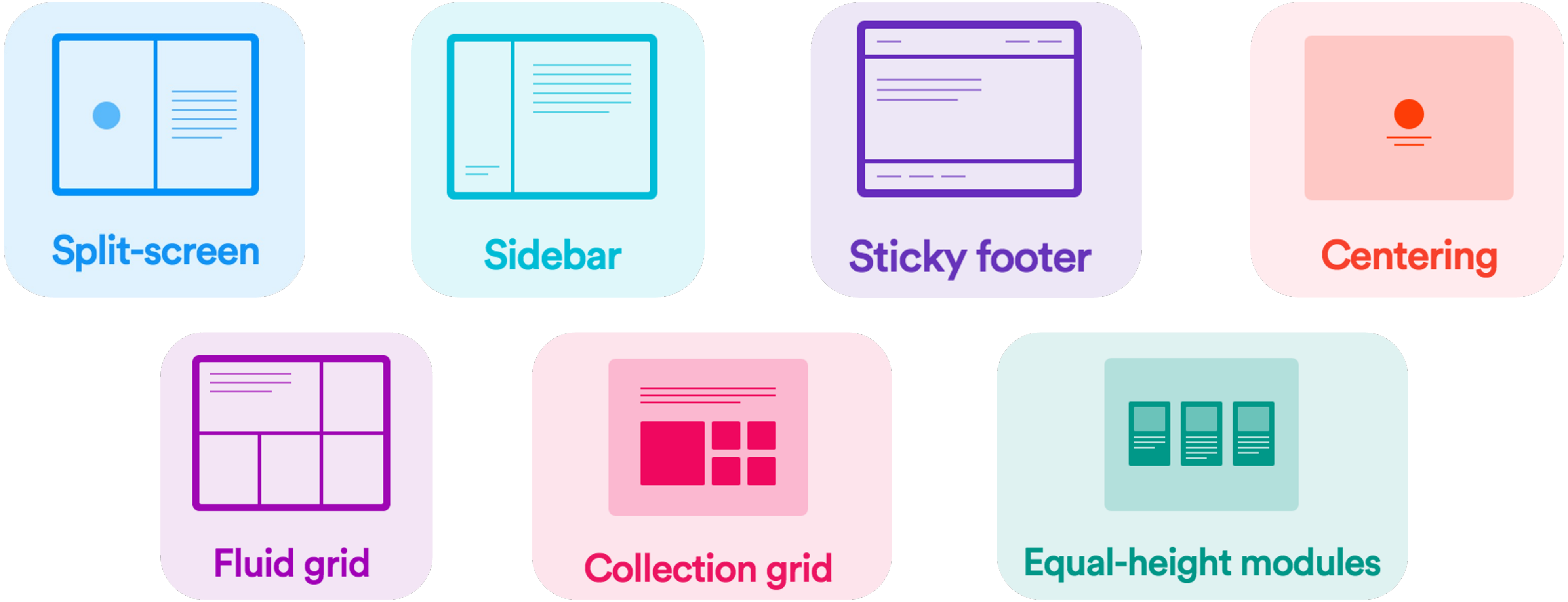


Flexbox

Introduction

Flex layout solves all sorts of problems

- Here are some examples of layouts that are easy to create with flex layout



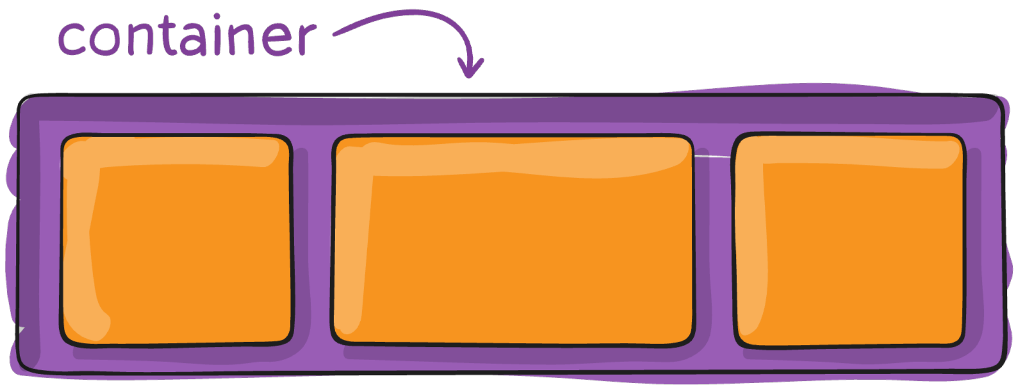
Flexbox

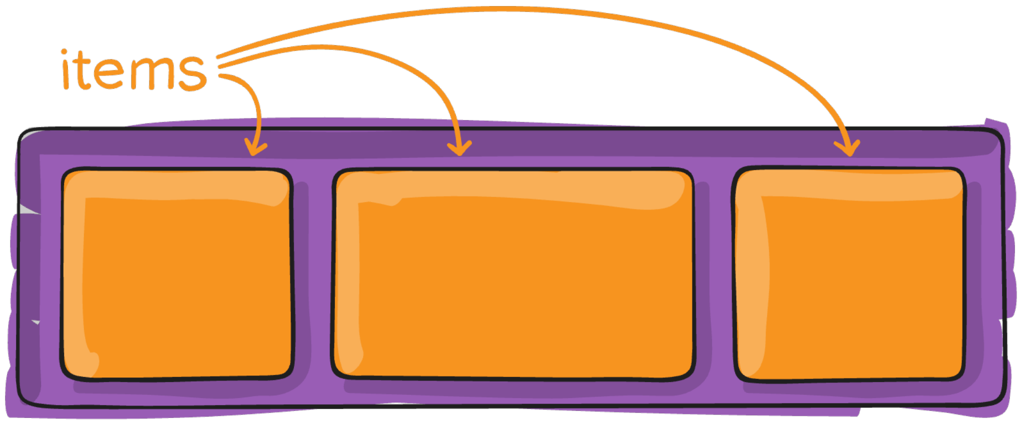
Flex Basics

Flex layouts are composed of:

- A **Flex container**, which contains one or more:
- **Flex item(s)**

You can then apply CSS properties on the flex container to dictate how the flex items are displayed.





Flexbox

Properties for the Parent (flex container)

`display`

This defines a flex container; inline or block depending on the given value. It enables a flex context for all its direct children.

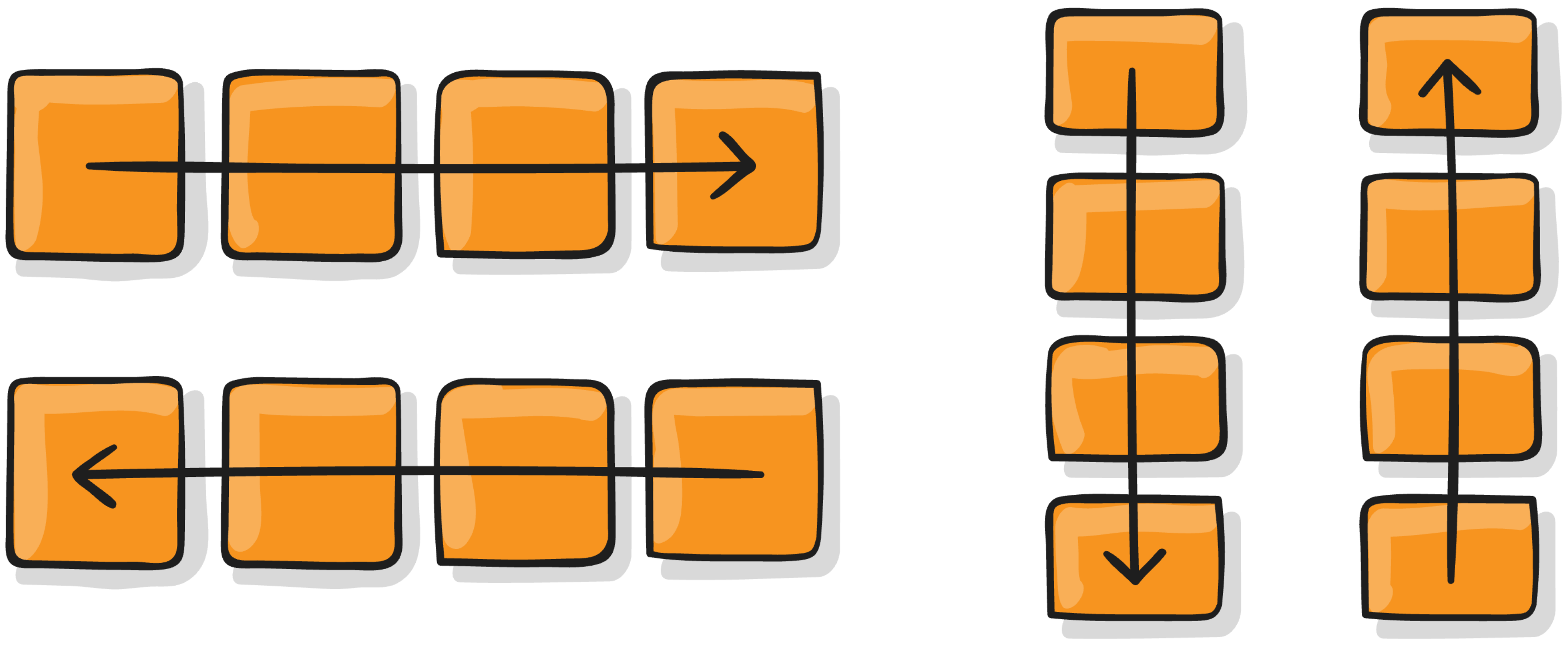
- Block container: `display: flex;` or
- Inline container: `display: inline-flex;`

```
.container {
   display: flex; /* or inline-flex */
}
```

Flexbox

Properties for the Parent (flex container)

`flex-direction`



Flexbox

Properties for the Parent (flex container)

`flex-direction`

This establishes the main-axis, thus defining the direction flex items are placed in the flex container.

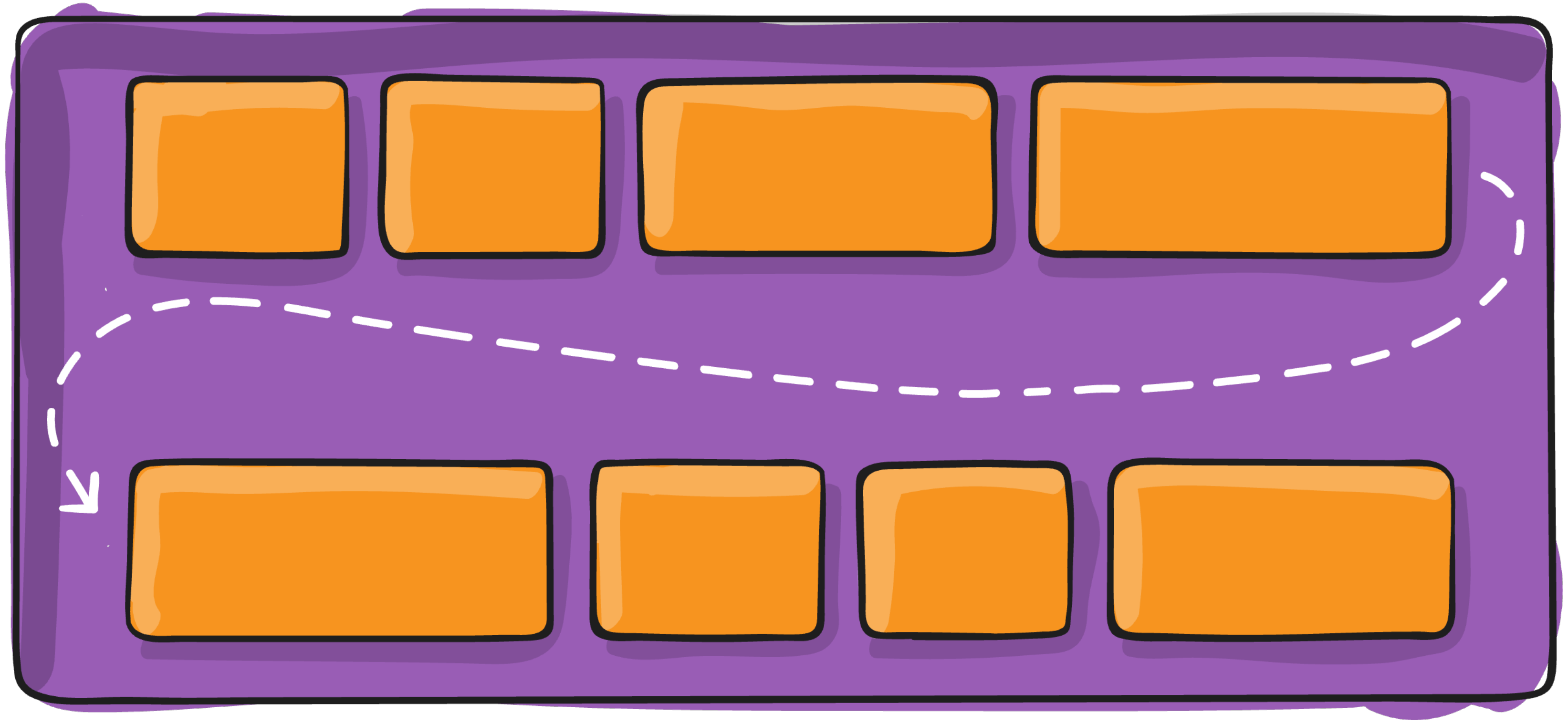
- `row` (default): left to right in `ltr`; right to left in `rtl`
- `row-reverse`: right to left in `ltr`; left to right in `rtl`
- `column`: same as `row` but top to bottom
- `column-reverse`: same as `row-reverse` but bottom to top

```css
.container {
   flex-direction: row | row-reverse | column | column-reverse;
}
```

Flexbox

Properties for the Parent (flex container)

`flex-wrap`



Flexbox

Properties for the Parent (flex container)

`flex-wrap`

By default, flex items will all try to fit onto one line. You can change that and allow the items to wrap as needed with this property.

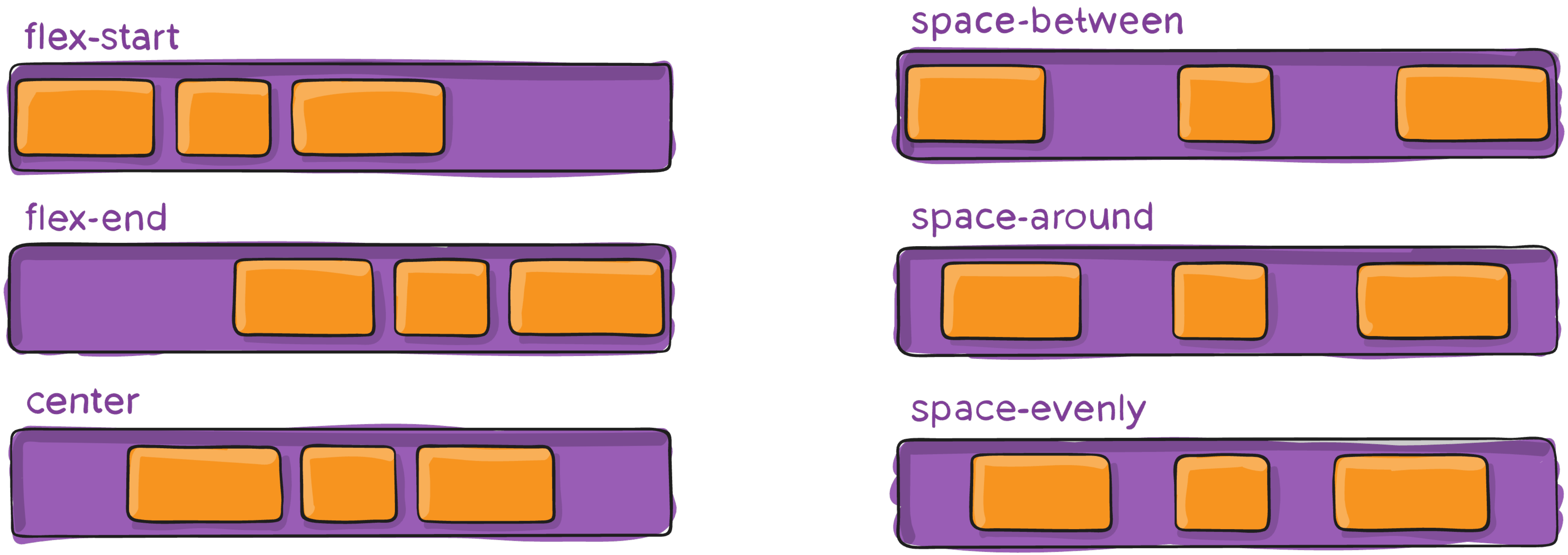
- `nowrap` (default): all flex items will be on one line
- `wrap`: flex items will wrap onto multiple lines, from top to bottom.
- `wrap-reverse`: flex items will wrap onto multiple lines from bottom to top.

```
.container {
   flex-wrap: nowrap | wrap | wrap-reverse;
}
```

Flexbox

Properties for the Parent (flex container)

`justify-content`



Flexbox

Properties for the Parent (flex container)

`justify-content`

This defines the alignment along the central axis. It helps distribute extra free space left over when all the flex items on a line are inflexible or flexible but have reached their maximum size.

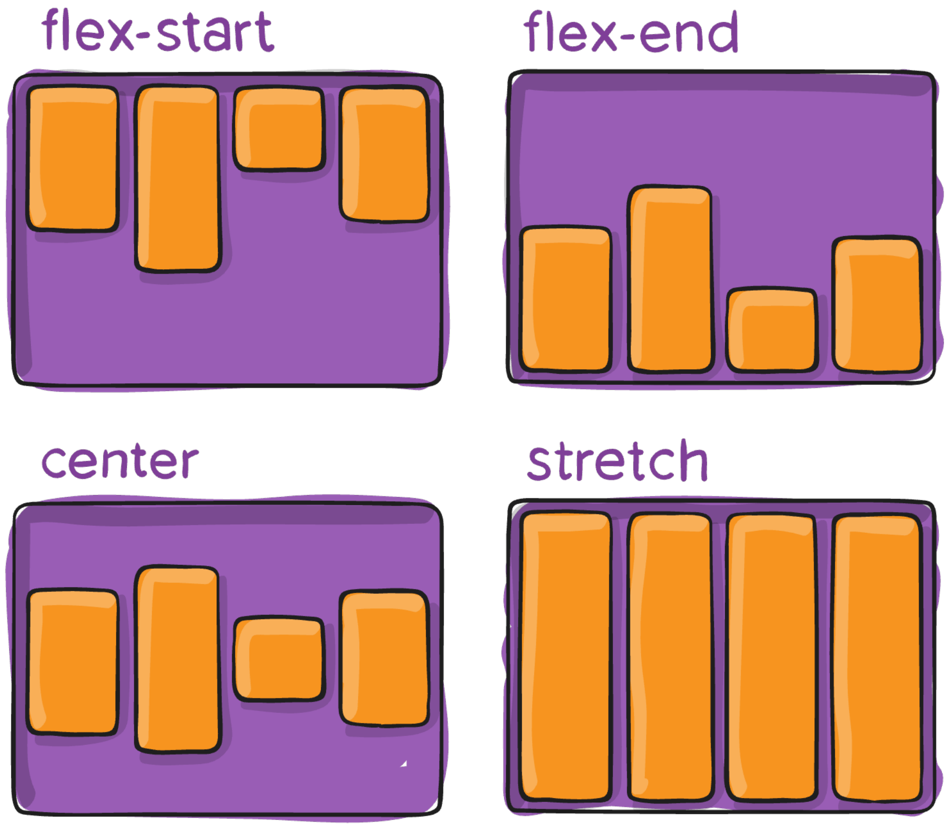
- `flex-start` (default): items are packed toward the start of the flex-direction.
- `flex-end`: items are packed toward the end of the flex-direction.
- `start`: items are packed toward the start of the `writing-mode` direction.

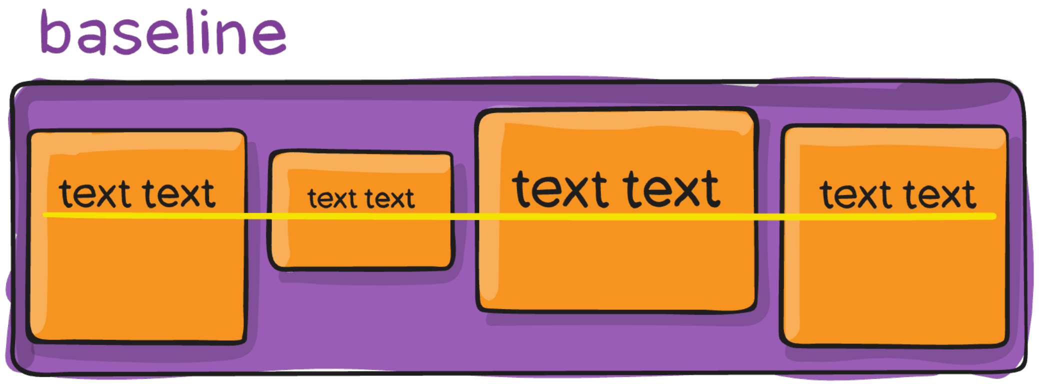
```
.container {
   justify-content: flex-start | flex-end | center | space-between | space-
   around | space-evenly | start | end | left | right ...;
}
```

Flexbox

Properties for the Parent (flex container)

`align-items`





Flexbox

Properties for the Parent (flex container)

`align-items`

This defines the default behavior for how flex items are laid out along the cross axis on the current line.

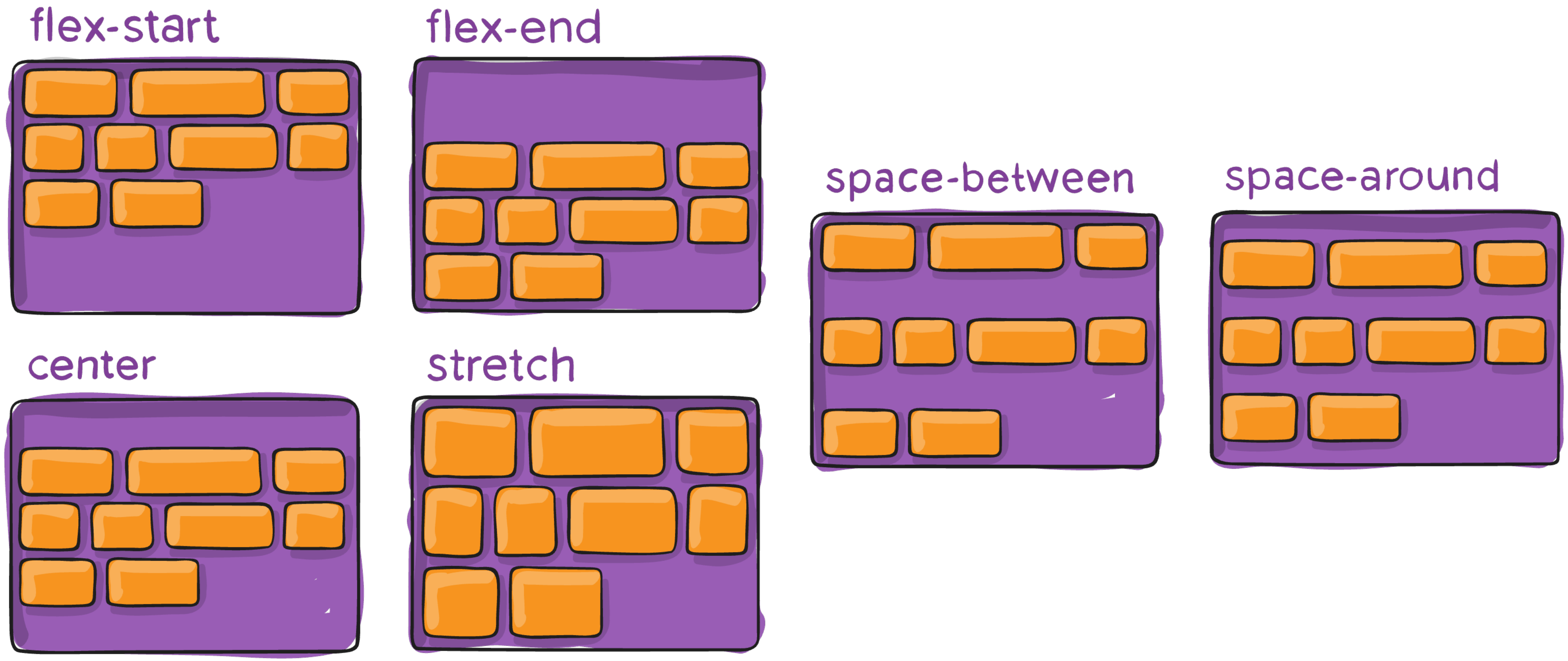
- `stretch` (default): stretch to fill the container (still respect min-width/max-width)
- `flex-start / start / self-start`: items are placed at the start of the cross-axis.
- `baseline`: items are aligned such as their baselines align

```
.container {
   align-items: stretch | flex-start | flex-end | center | baseline | first
   baseline | last baseline | start | end | self-start | self-end ...;
}
```

Flexbox

Properties for the Parent (flex container)

`align-content`



Flexbox

Properties for the Parent (flex container)

`align-content`

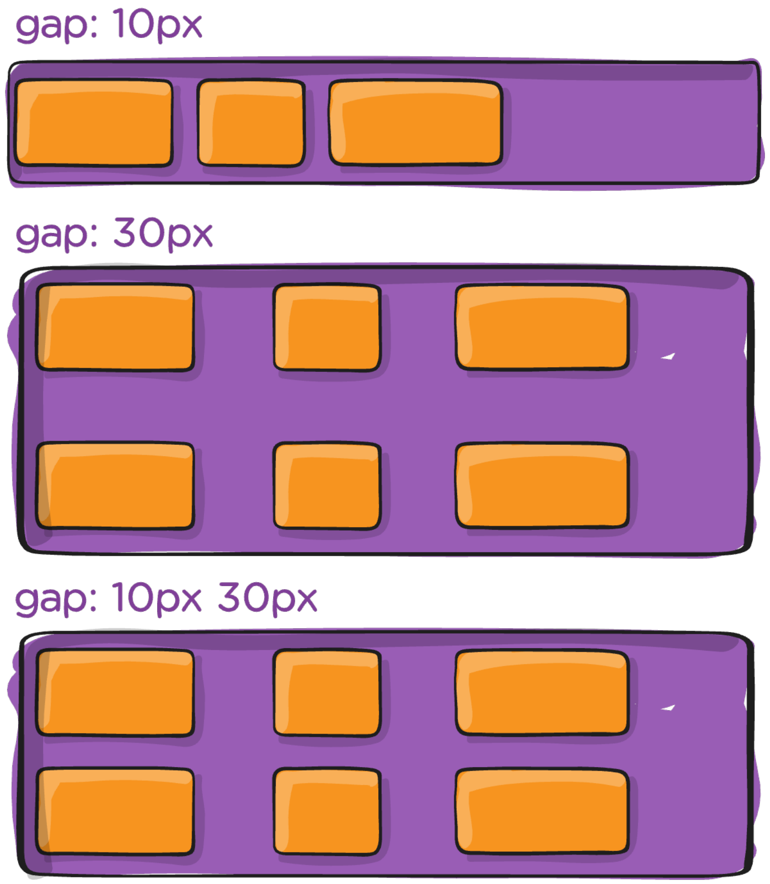
This aligns a flex container's lines within when there is extra space in the cross-axis, similar to how `justify-content` aligns individual items within the main axis.

```
.container {
   align-content: flex-start | flex-end | center | space-between | space-
   around | space-evenly | stretch | start | end | baseline | first baseline
   | last baseline ...;
}
```

Flexbox

Properties for the Parent (flex container)

`gap, row-gap, column-gap`



Flexbox

Properties for the Parent (flex container)

`gap, row-gap, column-gap`

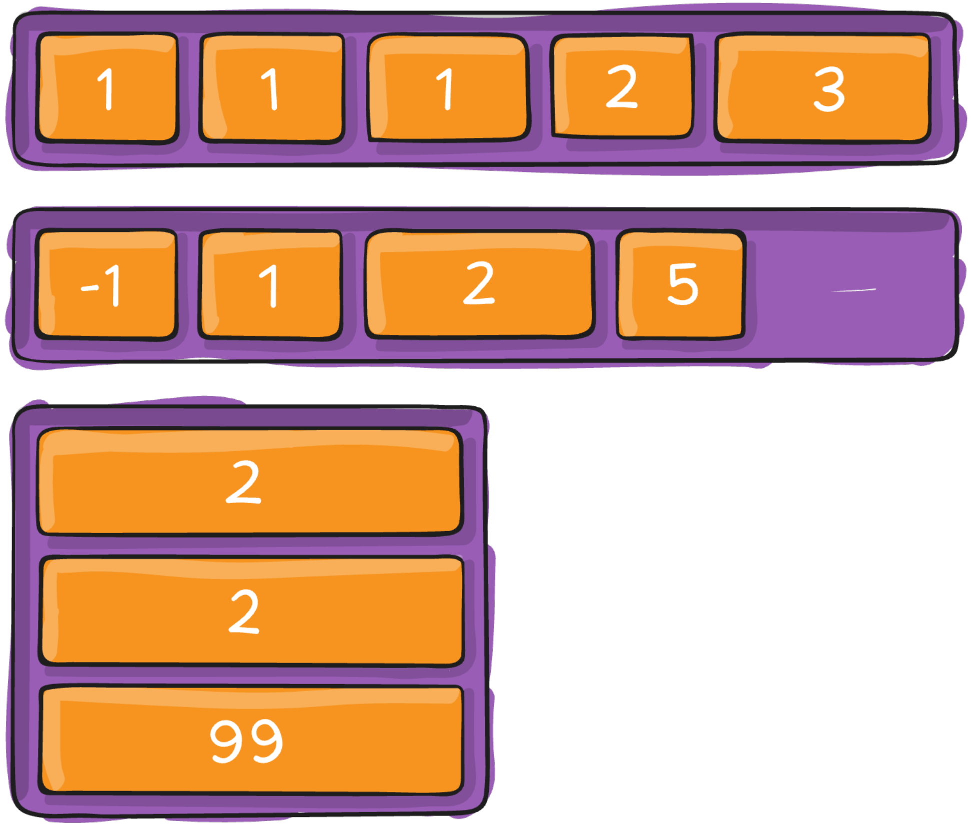
The gap property explicitly controls the space between flex items. It applies that spacing only between items not on the outer edges.

```
.container {
  display: flex;
  ...
  gap: 10px;
  gap: 10px 20px; /* row-gap column gap */
  row-gap: 10px;
  column-gap: 20px;
}
```

Flexbox

Properties for the Children (flex items)

`order`



Flexbox

Properties for the Children (flex items)

`order`

By default, flex items are laid out in the source order. However, the order property controls the order in which they appear in the flex container.

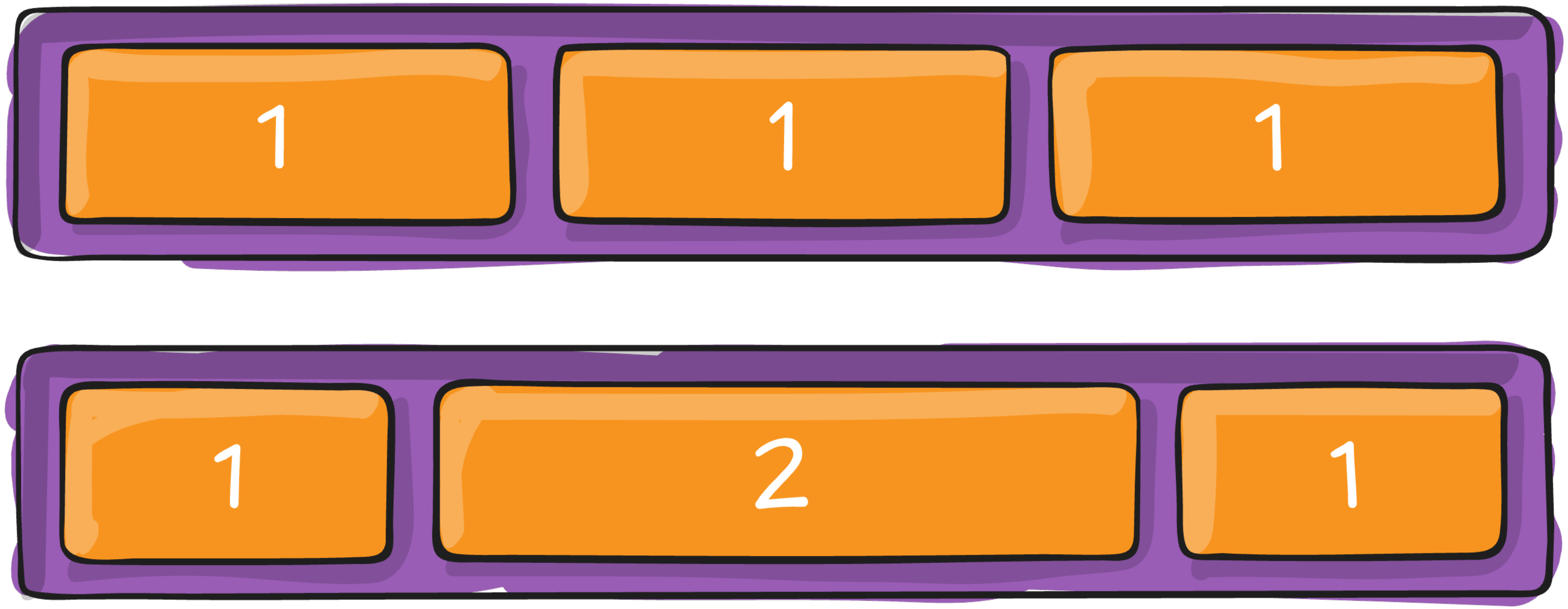
Items with the same order revert to source order.

```
.item {
  order: 5; /* default is 0 */
}
```

Flexbox

Properties for the Children (flex items)

`flex-grow or flex`



Flexbox

Properties for the Children (flex items)

`flex-grow`

This defines the ability for a flex item to grow if necessary. It accepts a unitless value that serves as a proportion.

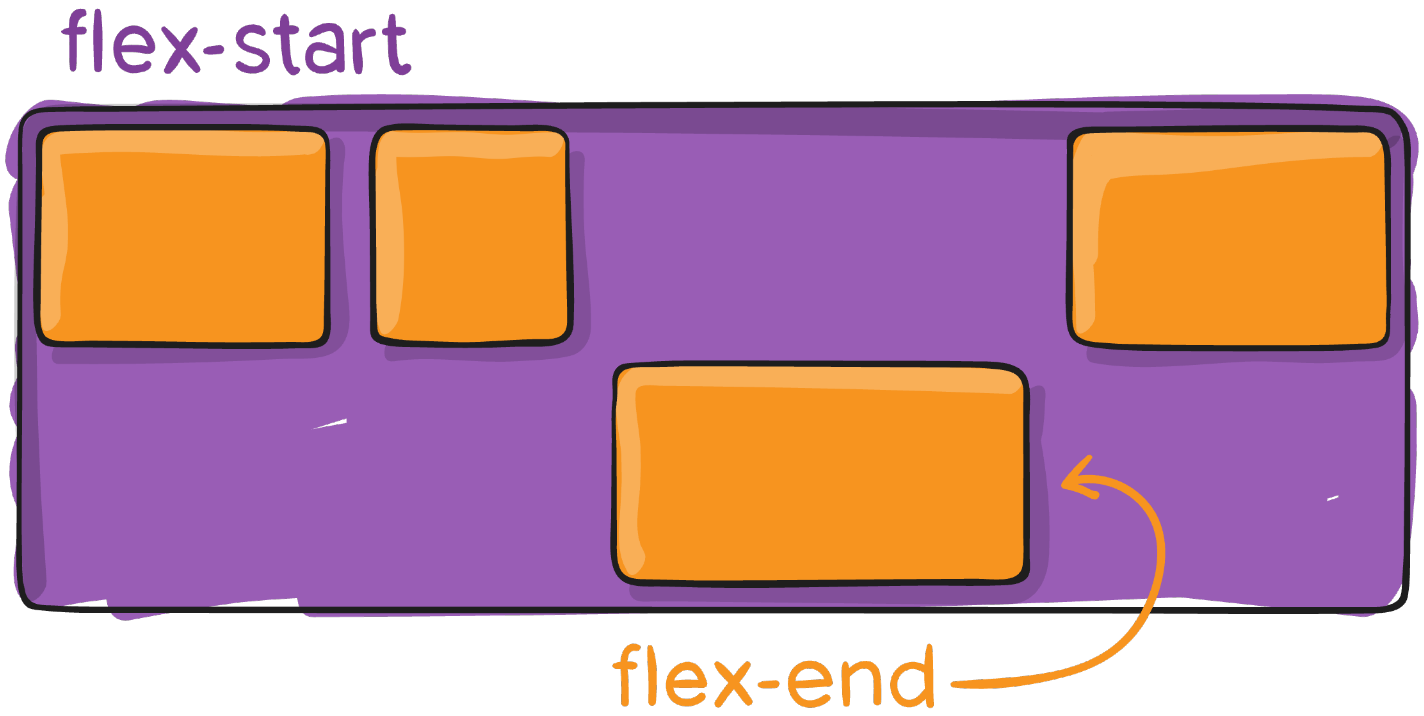
- If all items have flex-grow set to 1, the remaining space in the container will be distributed equally to all children.
- If one of the children has a value of 2, that child would take up twice as much of the space as either one of the others (or it will try, at least).

```css
.item {
  flex-grow: 4; /* default 0 */
}
```

Flexbox

Properties for the Children (flex items)

`align-self`



Flexbox

Properties for the Children (flex items)

`flex-grow`

This allows the default alignment (or the one specified by align-items) to be overridden for individual flex items.

```
.item {
  align-self: auto | flex-start | flex-end | center | baseline | stretch;
}
```

Flexbox

Practice

Practice 1

Small Practice Exercises

Bootstrap

What is Bootstrap?

- Open-source CSS framework developed by Twitter
- Helps build responsive, mobile-first websites
- Includes:
    - CSS for layout and design
    - JavaScript plugins for interactivity
    - Pre-built components (buttons, forms, navbars, etc.)

Bootstrap

Why use Bootstrap?

- Saves time with ready-made styles
- Ensures consistency across browsers and devices
- Built-in responsive grid system
- Active community and frequent updates

Bootstrap

Installing Bootstrap via CDN

A Content Delivery Network (CDN) hosts files (like CSS and JS) on fast global servers. Using a CDN means you don't need to download Bootstrap manually.

Why use the CDN version of Bootstrap?

- Easiest and fastest setup
- No installation required
- Gets the latest stable version
- Great for quick testing, small projects, or prototyping

Bootstrap

Installing Bootstrap via CDN

Include via CDN

When you only need to include Bootstrap's compiled CSS or JS, you can use jsDelivr. See it in action with our simple quick start, or browse the examples to jumpstart your next project. You can also choose to include Popper and our JS separately.

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.6/dist
```

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.6/dis
```

Bootstrap

Layout

Containers

- Containers are the most basic layout element in Bootstrap and are required when using our default grid system.
- Containers are used to contain, pad, and (sometimes) center the content within them.
- Bootstrap comes with three different containers:
    - `.container`, which sets a max-width at each responsive breakpoint
    - `.container-{breakpoint}`, which is width: 100% until the specified breakpoint
    - `.container-fluid`, which is width: 100% at all breakpoints

Bootstrap

Layout

Containers

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	X-Large ≥1200px	XX-Large ≥1400px
`.container`	100%	540px	720px	960px	1140px	1320px
`.container-sm`	100%	540px	720px	960px	1140px	1320px
`.container-md`	100%	100%	720px	960px	1140px	1320px
`.container-lg`	100%	100%	100%	960px	1140px	1320px
`.container-xl`	100%	100%	100%	100%	1140px	1320px
`.container-xxl`	100%	100%	100%	100%	100%	1320px
`.container-fluid`	100%	100%	100%	100%	100%	100%

Bootstrap

Layout

Grid system

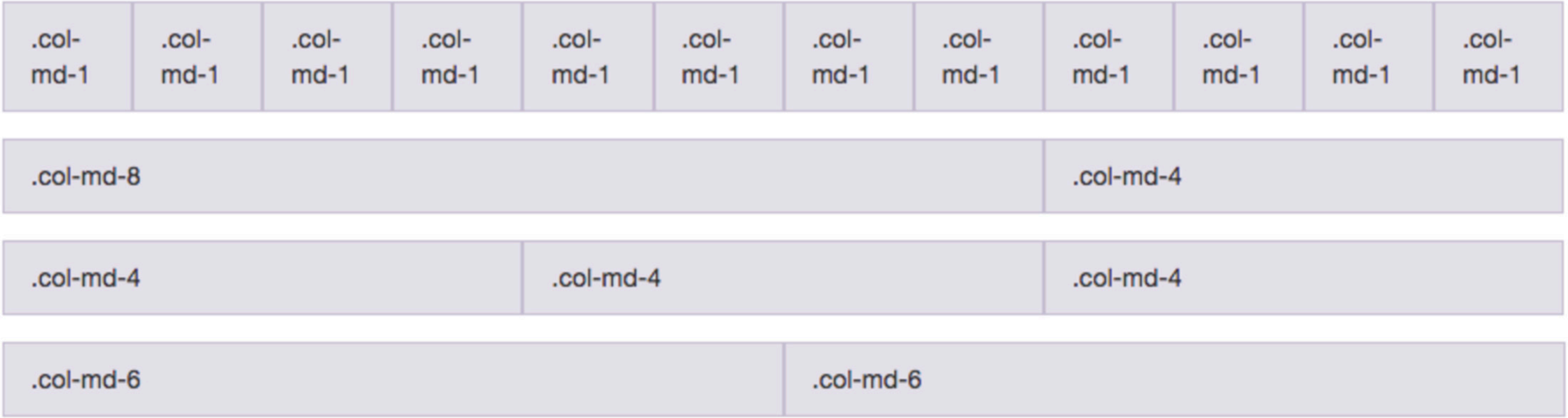
HTML

```html
<div class="container text-center">
  <div class="row">
    <div class="col">
      Column
    </div>
    <div class="col">
      Column
    </div>
    <div class="col">
      Column
    </div>
  </div>
</div>
```

Bootstrap

Layout

Grid system



Bootstrap

Components

What are Components in Bootstrap?

Pre-designed, reusable UI elements that help you build interactive and consistent web interfaces quickly.

Why use them?

- Save time – no need to design from scratch
- Responsive and accessible by default
- Easy to customize with utility classes
- Built-in interactivity with minimal JavaScript

Bootstrap

Components

Examples of Components

Flexbox & Bootstrap

Any questions?